

UNIVERSITÉ D'AVIGNON

MASTER 1 ILSSEN – AMS PROJET 1

Cahier des Charges

Extraction automatique de réseaux de personnages

Lotfi ABDALLAH
Wael MANSOURA

Année universitaire 2025–2026

Table des matières

1	Contexte et Objectifs du Projet	2
1.1	Contexte	2
1.2	Objectifs	2
2	Périmètre du Projet	2
2.1	Fonctionnalités attendues	2
2.2	Technologies utilisées	2
3	Public Cible	3
3.1	Profil des utilisateurs	3
3.2	Besoins des utilisateurs	3
4	Scénarios de Tests Utilisateurs	3
4.1	Objectifs des tests	3
4.2	Méthodologie	3
5	Critères de Réussite et Évaluations	3
5.1	Indicateurs de performance (KPI)	3
5.2	Feedback des utilisateurs / professeurs	3
6	Contraintes et Risques	3
6.1	Contraintes techniques	3
6.2	Risques potentiels	4
7	Planning Prévisionnel	4
7.1	Phases du projet	4
7.2	Livrables	4
8	Équipe du Projet	4
9	Annexes	4
9.1	Architecture du système	4
9.2	Description des scripts Python	6

1. Contexte et Objectifs du Projet

1.1. Contexte

L'extraction automatique de réseaux de personnages à partir de textes narratifs est une tâche complexe qui combine des techniques de traitement du langage naturel (NLP) et la modélisation graphique.

Ce projet vise à développer un système capable :

- d'identifier les personnages dans un texte,
- de détecter leurs interactions,
- de construire un réseau représentatif des relations entre ces personnages.

1.2. Objectifs

Les objectifs principaux du projet sont :

- Développer un algorithme d'extraction de personnages à partir de textes narratifs.
- Construire des listes :
 - **L** – toutes les entités nommées candidates,
 - **LP** – les personnages (personnes),
 - **LL** – les lieux.
- Détecter les cooccurrences de personnages en utilisant la proximité textuelle.
- Construire un réseau de personnages basé sur les interactions détectées.

2. Périmètre du Projet

2.1. Fonctionnalités attendues

1. Extraction des entités nommées.
2. Filtrage des personnes (liste **LP**).
3. Filtrage des lieux (liste **LL**).
4. Regroupement d'alias pour chaque personnage.
5. Génération du graphe avec **NetworkX** pour chaque chapitre.
6. Détection des interactions entre les personnages.
7. Construction et visualisation du réseau de personnages.
8. Export CSV des résultats pour plusieurs chapitres.

2.2. Technologies utilisées

Catégorie	Outils / Technologies
Langage de programmation	Python
Bibliothèques NLP	spaCy, Stanza, Flair
Visualisation	NetworkX, Matplotlib
Filtrage	<code>antidict.txt</code> (entités non pertinentes)

TABLE 1 – Technologies et outils utilisés

3. Public Cible

3.1. Profil des utilisateurs

- Étudiants en informatique.
- Chercheurs en linguistique computationnelle.
- Développeurs en traitement du langage naturel.

3.2. Besoins des utilisateurs

- Outils efficaces pour l'extraction de réseaux de personnages.
- Visualisation claire des réseaux de personnages.

4. Scénarios de Tests Utilisateurs

4.1. Objectifs des tests

- Valider que les outils d'extraction identifient correctement les personnages importants.
- Vérifier la pertinence du regroupement d'alias.
- Évaluer la cohérence des co-occurrences détectées.

4.2. Méthodologie

1. Sélectionner un ensemble de textes narratifs variés.
2. Appliquer les outils d'extraction développés.
3. Comparer les résultats obtenus avec des annotations manuelles.

5. Critères de Réussite et Évaluations

5.1. Indicateurs de performance (KPI)

- Taux de précision de l'extraction des personnages.
- Score de précision du graphe de personnages par chapitre sur le classement Kaggle.
- Temps de génération du graphe pour chaque chapitre.

5.2. Feedback des utilisateurs / professeurs

- Utilisation de plusieurs modèles NLP pour extraire les personnages.
- Méthode de **vote** entre les modèles pour identifier les personnages corrects, plutôt que d'utiliser l'intersection ou l'union simple des résultats.

6. Contraintes et Risques

6.1. Contraintes techniques

- Limites des modèles NLP pour la reconnaissance des entités nommées.
- Gestion des ambiguïtés dans les noms des personnages.

- Performance et scalabilité pour traiter de grands textes narratifs.
- Fusion des résultats de plusieurs modèles NLP pour améliorer la précision.

6.2. Risques potentiels

- Mauvaise détection d'entités à cause du style littéraire.
- Difficulté à atteindre un taux de précision élevé.
- Problèmes de performance avec de grands volumes de texte.

7. Planning Prévisionnel

7.1. Phases du projet

Phase	Durée
Analyse et préparation du corpus	2 semaines
Conception de l'extraction NER et des listes L/LP/LL	3 semaines
Identification des cooccurrences	2 semaines
Production des graphes	2 semaines
Ajustements finaux et rédaction du rapport	1 semaine
Total	10 semaines

TABLE 2 – Planning prévisionnel du projet

7.2. Livrables

- Listes L, LP, LL.
- Réseaux de personnages par chapitre dans un fichier CSV.
- Cahier des charges finalisé.

8. Équipe du Projet

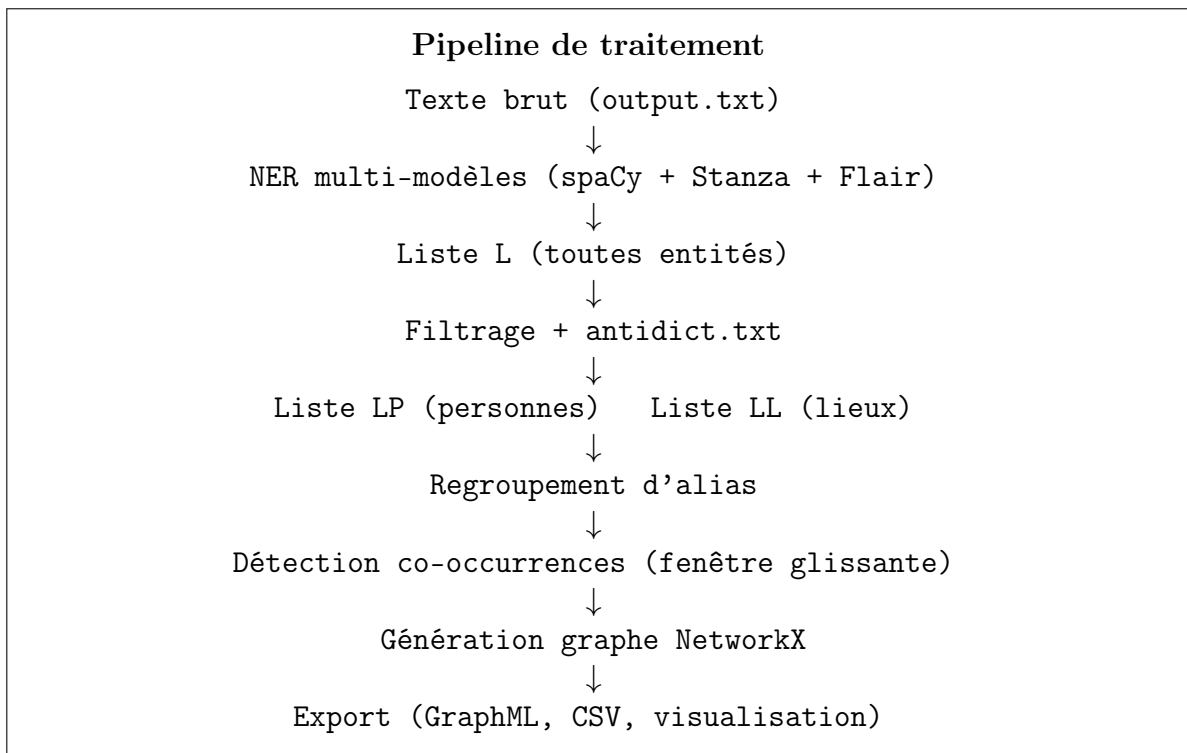
Membre	Responsabilités
Lotfi ABDALLAH	Développement NLP, extraction NER, listes L/LP/LL
Wael MANSOURA	Détection co-occurrences, génération graphes

TABLE 3 – Répartition des responsabilités

9. Annexes

9.1. Architecture du système

Le pipeline d'extraction de réseaux de personnages suit l'architecture suivante :



Le notebook `main.ipynb` orchestre l'ensemble du pipeline et permet une exécution interactive étape par étape.

9.2. Description des scripts Python

Fichier	Rôle
<code>main.ipynb</code>	Notebook principal – orchestre le pipeline complet : chargement des données, extraction NER, filtrage, alias, co-occurrences, génération et visualisation des graphes. Point d'entrée interactif du projet.
<code>nlp_main.py</code>	Script d'orchestration en ligne de commande. Appelle les modules dans l'ordre pour produire les listes L, LP, LL.
<code>nlp_extract_characters.py</code>	Extraction des entités nommées avec spaCy. Fonctions <code>extract_entities()</code> , <code>count_entities()</code> , <code>filter_persons()</code> , <code>filter_locations()</code> .
<code>nlp_multi_ner.py</code>	NER multi-modèles (spaCy + Stanza). Implémente la méthode de vote via <code>ensemble_entities()</code> pour fusionner les résultats des différents modèles.
<code>nlp_aliases.py</code>	Regroupement d'alias : normalisation des noms, détection des variantes (ex : "Hari" → "Hari Seldon"), fusion des comptages.
<code>nlp_cooccurrence.py</code>	Détection des co-occurrences entre personnages dans une fenêtre de mots configurable (<code>distance_max</code>).
<code>nlp_graph.py</code>	Génération du graphe NetworkX, sauvegarde en GraphML, visualisation statique avec Matplotlib.
<code>nlp_visualize_web.py</code>	Visualisation interactive du graphe en HTML avec PyVis.
<code>nlp_create_submission.py</code>	Traitement par chapitre et génération du fichier <code>submission.csv</code> pour Kaggle.
<code>nlp_utils.py</code>	Fonctions utilitaires : lecture de fichiers, chargement de l'anti-dictionnaire.

TABLE 4 – Description des modules Python du projet